

■ Supermarket Operations Agent

Comprehensive Technical Architecture & Deep Codebase Analysis Report

Project Name:	AI SmartMart (Supermarket Operations Agent)	Primary Language:	Python 3.11+
Interface:	Telegram Conversational Bot & REST Webhook	Database:	PostgreSQL 16 + SQLAlchemy ORM
LLM Engine:	LiteLLM (Gemini 1.5 / OpenAI / Claude)	Framework:	FastAPI 0.110+ & Uvicorn

1. Executive Summary & Core Value Proposition

AI SmartMart is an autonomous conversational AI agent tailored specifically for Indian supermarket and Kirana store operators. Unlike legacy point-of-sale (POS) systems that rely on rigid button menus or static UI trees, AI SmartMart operates entirely through natural language over Telegram. Store managers can manage stock inwarding, multi-item customer billing, GST tax calculation, credit ledgers (Khata), and sales analytics simply by sending plain text, voice notes, or invoice images.

The project addresses the core operational friction of retail store managers by pairing model-orchestrated tool calling with deterministic SQL business logic, guaranteeing zero inventory hallucination, strict financial correctness, and atomic oversell protection.

2. System Architecture & Control Loop

Rather than forcing complex graph state machines or hardcoded if/else routing, the system implements a **Tool-Calling Agent Harness** combined with **Atomic Domain Services**.

The Agent Control Loop Execution Flow:

- Observation & Ingestion:** Incoming Telegram messages (Text, Voice, or Photo) are received via Telegram Webhook (FastAPI) or Polling.
- Fast Intent Routing (AgentOrchestrator):** The engine evaluates regex patterns for ultra-fast local intent matching (e.g. quick stock checks, direct bill additions) to avoid unnecessary LLM latency.
- LLM Tool Selection & Reasoning:** For complex or multi-turn queries, LiteLLM dispatches requests to the LLM (e.g. Gemini 1.5 Flash) with registered function schemas.
- Atomic Tool Execution:** Deterministic domain services execute against PostgreSQL in transaction boundaries.
- Feedback Loop & Response:** Tool results are fed back into context, yielding formatted responses in INR (■) or downloadable PDF/PPTX artifacts.

3. Solutions to Key Engineering Challenges ('The 9 Hard Parts')

#	Challenge	Engineering Solution in AI SmartMart
1	Grounding	Prices, stock levels, GST slabs, and SKUs are fetched strictly from PostgreSQL via database queries. Prompt rules explicitly forbid hallucinating unlisted items.
2	Oversell Guard	Enforced at SQL layer: UPDATE products SET quantity = quantity - :qty WHERE id = :id AND quantity >= :qty. Zero negative stock possible.
3	GST Correctness	Deterministic Python Decimal calculator for intra-state 50/50 CGST/SGST split, line-item HALF_UP rounding, and HSN tax mapping.
4	Multi-Turn Bills	Active carts are held in bill_drafts table per chat_id, allowing mid-bill updates ('add 2 maggi', 'drop sugar') before final checkout.
5	Idempotency	Telegram update_id values are recorded in processed_messages to drop duplicate webhooks or redelivered polling messages.
6	Concurrency Protection	Atomic PostgreSQL row locking ensures concurrent checkouts on the same SKU do not corrupt stock values.
7	Commercial Guardrails	Guards against below-cost selling, accidental inventory wipes, and invalid negative Khata ledger balances.
8	Real Artifacts	Generates official GST Tax Invoice PDFs on-the-fly (ReportLab) and PowerPoint weekly business reports (python-pptx).
9	Cross-Session Memory	Store preferences (GSTIN, default payment method, store name) persist in owner_preferences table across sessions.

4. Codebase Architecture & Key Modules

The codebase is structured cleanly in the `src/` directory, separating API routes, ORM models, business logic services, schemas, and LLM agent orchestration:

File Path	Responsibility & Key Functions
<code>src/main.py</code>	FastAPI application entry point. Configures Telegram bot handlers (<code>/start</code> , <code>/updatestock</code> , <code>/billing</code> , <code>/khata</code> , <code>/analytics</code> , <code>/lowstock</code>), webhook route (<code>/webhook</code>), and APScheduler background tasks.
<code>src/agent.py</code>	Contains AgentOrchestrator, local regex intent matcher, and LiteLLM tool definition array (<code>check_stock</code> , <code>update_stock</code> , <code>add_bill_item</code> , <code>finalize_bill</code> , <code>record_khata_transaction</code> , etc.).
<code>src/services.py</code>	Contains atomic core business logic services: • InventoryService: stock checks, restock txns, low stock alerts. • BillingService: draft bill management, tax calculation, checkout. • KhataService: customer credit balance and repayment entries. • AnalyticsService: sales summary, GST reports, top movers.
<code>src/models.py</code>	SQLAlchemy 2.0 ORM Declarative Models: <code>Product</code> , <code>InventoryTransaction</code> , <code>Customer</code> , <code>KhataTransaction</code> , <code>Bill</code> , <code>BillItem</code> , <code>BillDraft</code> , <code>OwnerPreference</code> , <code>ProcessedMessage</code> .
<code>src/schemas.py</code>	Pydantic v2 schemas for API data serialization and validation (<code>ProductSchema</code> , <code>BillSchema</code> , <code>BillItemSchema</code> , <code>CustomerSchema</code>).
<code>src/database.py</code>	Database connection engine setup and session lifecycle generator (<code>get_db</code>).
<code>alembic/</code> & SQL migrations	Database schema version control scripts and SQL migration files (<code>V1__init_schema.sql</code> , <code>V2__seed_ai_smart_mart_products.sql</code>).

5. Relational Data Model & Schema Overview

The relational schema is engineered for high data integrity, strict auditability, and optimal query indexing. Below is an overview of the core tables:

Table	Key Columns & Types	Description / Business Role
<code>products</code>	<code>id</code> (PK), <code>sku</code> (UQ), <code>name</code> , <code>normalized_name</code> , <code>unit</code> , <code>cost_price</code> , <code>selling_price</code> , <code>mrp</code> , <code>quantity</code> , <code>gst_rate</code> , <code>hsn_code</code>	Stores complete SKU master catalog, stock levels, pricing, and GST slabs.
<code>inventory_transactions</code>	<code>id</code> (PK), <code>product_id</code> (FK), <code>transaction_type</code> , <code>quantity_change</code> , <code>balance_after</code> , <code>created_at</code>	Immutable ledger of all stock movements (STOCK_IN, SALE, RETURN, ADJUSTMENT).
<code>customers</code>	<code>id</code> (PK), <code>name</code> , <code>normalized_name</code> , <code>phone</code> , <code>current_credit_balance</code>	Customer master records and current outstanding credit balance.
<code>khata_transactions</code>	<code>id</code> (PK), <code>customer_id</code> (FK), <code>transaction_type</code> , <code>amount</code> , <code>balance_after</code> , <code>notes</code>	Audit trail of credit purchases and payment settlements.
<code>bills & bill_items</code>	<code>id</code> (PK), <code>bill_number</code> (UQ), <code>total_taxable</code> , <code>total_cgst</code> , <code>total_sgst</code> , <code>grand_total</code> , <code>payment_mode</code>	Finalized sales invoices and itemized line items with exact tax breakdown.
<code>bill_drafts</code>	<code>chat_id</code> (PK), <code>draft_json</code> , <code>updated_at</code>	Transient state storage for multi-turn cart building before checkout.

6. Deployment & Containerization Setup

The project is fully containerized using Docker and Docker Compose for seamless single-command deployment across local development and production servers:

- **Docker Environment:** Multi-stage builds utilizing Python 3.11-slim base image.
- **Database Container:** Managed PostgreSQL 16 container pre-configured with healthchecks.
- **Service Orchestration:** docker-compose.yml links the FastAPI agent service with PostgreSQL, exposing port 8080.
- **Background Scheduler:** Integrated APScheduler automatically executes low-stock morning alerts and weekly business report generation.

7. Codebase Audit Findings & Recommendations

During our full codebase analysis, the following observations and improvements were identified:

■ Code Integrity Maintenance:

- Cleaned Git merge conflict markers in configuration files (pyproject.toml and src/main.py) to ensure continuous build stability.
- Standardized dependency declarations using Hatch build system.

■ Production Enhancements:

- **Whisper Voice Integration:** Connect native OpenAI Whisper or local Faster-Whisper to replace simulated voice note processing in src/main.py.
- **Vision OCR Integration:** Connect Google Gemini Vision / GPT-4o Vision to parse paper supplier invoices directly into stock intake.
- **Database Connection Pooling:** Fine-tune SQLAlchemy pool sizes for high-frequency concurrent store webhooks.

Report Generated On: September 2026 | Author: Antigravity AI Engineer